Robótica II	Universidad Nacional de Asunción	
Ing. Mecatrónica	Campus San Lorenzo	
Semestre X	Facultad de Ingeniería	

μTP
Robótica Social
Cabeza social reactiva de 3 GDL con visión artificial
GRUPAL (máx. 5 personas)

1. Descripción

El micro trabajo práctico (μTP) consiste en diseñar, construir y programar una **cabeza robótica social** con ojos, montada sobre un cuello de **3 grados de libertad** (guiñada/*yaw*, cabeceo/*pitch* y alabeo/*roll*), que permita orientar la mirada. La cabeza lleva una **cámara solidaria** (en uno de los ojos o entre ambos, a la altura de la nariz), de modo que "mirar" a alguien equivale a centrar a esa persona en la imagen. Un algoritmo de **inteligencia artificial** procesa el video, detecta personas, rostros, gestos y objetos de interés, y el robot responde con movimientos de cabeza y con sonidos generados por un **buzzer pasivo modulado en frecuencia**.

El foco del trabajo no está sólo en que cada comportamiento funcione por separado, sino en que el robot sea **reactivo y robusto**: debe responder rápido, con movimientos suaves, y **nunca superponer acciones contradictorias** (por ejemplo, dos sonidos a la vez, o seguir a una persona mientras intenta asentir). Para ello se exige una arquitectura de comportamientos con arbitraje explícito.

2. Reglamento

1. **Integrantes.** El trabajo debe ser realizado en grupos de **máximo 5 personas**. Todos los integrantes deben estar presentes en la demostración y poder responder preguntas sobre cualquier parte del sistema. La nota individual podrá ajustarse según la defensa. La contribución de cada miembro del grupo debe ser clara (un set de features, detccciones, comportamientos, control, etc.)
2. Las características, comportamientos y reacciones del robot son referenciales. **Se valora que el grupo personalice el comportamiento y funciones del robot.**
3. **Puntaje.** El μTP vale **15 puntos** en total (3 pts directos y 12 pts adicionales, que pueden sobrescribir hasta 3 tests a seleccionar).
4. **Demo obligatoria.** Sin demostración funcional del prototipo físico no se asigna puntaje en el Test. Una simulación o un video no reemplazan a la demo.
5. **Fecha de entrega.** El μTP se entrega y se defiende en la **fecha del Segundo Parcial**. La documentación debe subirse al aula virtual antes del inicio de la demo.
6. **Herramientas.** Se permite el uso de modelos preentrenados y librerías de terceros (OpenCV, YOLO, etc.), indicando claramente la fuente. La lógica de comportamiento, el arbitraje y el control de la cabeza deben ser desarrollo propio del grupo.

7. **Integridad.** La copia total o parcial entre grupos implica la anulación del trabajo para todos los grupos involucrados. Se puede utilizar IA para el desarrollo del trabajo, pero se debe comprender el proceso y el resultado obtenidos, sobre los cuales se podrá preguntar en detalle.

3. Objetivos

Objetivo general

Desarrollar una cabeza robótica social de 3 GDL que, mediante visión artificial basada en IA, perciba personas y señales sociales y responda con comportamientos de mirada, gestos de cabeza y sonidos, de forma reactiva, coherente y robusta.

Objetivos específicos

1. Diseñar y construir un mecanismo cuello–cabeza de 3 GDL (yaw, pitch, roll) con cámara embarcada y controlarlo con movimientos suaves y dentro de límites seguros.
2. Implementar un sistema de percepción con IA capaz de detectar personas y/o objetos.
3. Implementar un lazo de control visual (*visual servoing*) que mantenga el contacto visual (*eye contact*) con una persona en movimiento.
4. Diseñar una arquitectura basada en comportamientos con prioridades, gestión de recursos, y filtrado temporal que evite la superposición de acciones y las oscilaciones.
5. Generar sonidos expresivos (melodías, silbidos, "voz" tipo R2D2) mediante modulación de frecuencia de un buzzer pasivo, de forma no bloqueante (es decir, que sigan funcionando la visión artificial y el mecanismo de la cabeza).
6. Documentar técnicamente el desarrollo y validarlo mediante una demostración funcional.

4. Descripción del sistema

4.1. Hardware mínimo

- **Cabeza con ojos** (impresa en 3D, cartón, MDF, etc.). Los ojos pueden ser fijos o con párpados/LEDs (opcional). Debe ser evidente hacia dónde "mira" el robot.
- **Cuello de 3 GDL** con servomotores. Rangos mínimos sugeridos: yaw $\pm 70°$, pitch $-20°/+30°$, roll $\pm 20°$.

- **Cámara** solidaria a la cabeza (webcam USB, cámara de ESP32-CAM, cámara de Raspberry Pi, etc.).
- **Buzzer pasivo** excitado con señal PWM de frecuencia variable (`tone()`, LEDC del ESP32, etc.).
- **Microcontrolador** para servos y buzzer (Arduino, ESP32, etc.) y **equipo de procesamiento** para la IA (PC, notebook, Raspberry Pi, Jetson), comunicados por serie, Wi-Fi o similar.

4.2. Arquitectura de software

La Figura 1 muestra la arquitectura de referencia. La percepción produce *eventos/predicados* filtrados en el tiempo (no superponer reacciones); un modelo del mundo mantiene las personas rastreadas; los comportamientos proponen acciones y un **árbitro** decide cuál(es) se ejecutan según prioridad y recursos (cuello y buzzer).

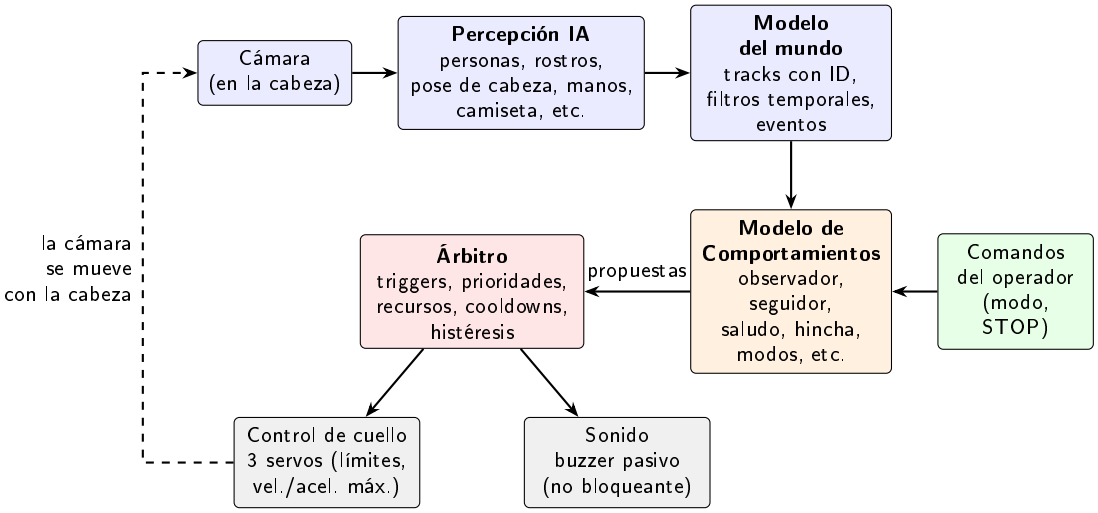


Figura 1: Arquitectura de referencia del sistema.

5. Tareas

T1. Plataforma mecatrónica y control de bajo nivel

a) Construir la cabeza y el cuello de 3 GDL; definir una **posición de reposo** (`HOME`) con la mirada al frente.

b) Implementar límites articulares por software y limitación de velocidad y aceleración de cada servo (trayectorias suaves, sin golpes ni vibraciones).

c) Implementar un protocolo de comunicación entre el equipo de procesamiento y el microcontrolador (p. ej., `J yaw pitch roll`, `S id_sonido`, `STOP`).

T2. Percepción

Implementar, con IA, la detección de los siguientes **eventos**, cada uno con una definición operativa documentada (por ejemplo, umbral de confianza y filtro temporal del tipo "N de M cuadros"):

- `PERSONA`: presencia de una o más personas, con **seguimiento por ID** (*tracking*) entre cuadros.
- `ME_MIRA`: la persona mira al robot (p. ej., orientación estimada del rostro dentro de $\pm 15°$ respecto de la cámara, sostenida $\geq 0{,}5$ s).
- `SALUDO`: la persona saluda con la mano (p. ej., mano por encima del hombro con oscilación lateral de al menos 2 ciclos).
- `CAMISETA_OLIMPIA` / `CAMISETA_CERRO`: clasificación de la camiseta a partir de la región del torso (clasificador entrenado, reglas de color/patrón, o combinación). Debe minimizarse el falso positivo: una remera lisa roja o azul **no** es de Cerro Porteño, ni una remera blanca lisa es de Olimpia.
- `SALIENDO`: la persona seguida está por abandonar el campo de visión del robot (cerca del borde y alejándose, o fuera del rango articular del cuello).

Estos son, básicamente, los algoritmos que nos permiten construir el modelo del mundo: si ocurren estas entidades (con sus características temporales y de confianza) y dónde se encuentran.

T3. Control de la mirada (eye contact)

a) Implementar un lazo de control visual que lleve el rostro de la persona objetivo al centro de la imagen (o al punto que corresponda a los ojos del robot), actuando sobre yaw y pitch.

b) Incluir zona muerta, saturación y suavizado, de forma que la mirada sea estable (sin oscilaciones) cuando la persona está quieta y la siga con fluidez cuando se mueve.

c) Ante pérdida momentánea del objetivo (< 1 s) mantener o predecir la mirada; si la pérdida persiste, liberar el objetivo. Al liberar el objetivo, se puede hacer un movimiento de desaprobación con la cabeza.

d) Si el objetivo permanece quieto (tipo cara a cara), el robot puede inclinar la cabeza (roll), es decir, lanzar ese comportamiento.

e) Si no detecta nada, puede pasar a un estado de aburrimiento.

Las acciones de control (seguir a la persona —yaw, pitch, roll—, detenerse, movimiento de aburrimiento, desaprobación, etc.) deben definirse en el módulo "Árbitro", que sería nuestro controlador.

T4. Selección del foco de atención

Ya tenemos el mecanismo y el seguidor de personas; ahora resolvamos la gestión del foco de atención si hay muchas personas.

En nuestra arquitectura, esto también forma parte del módulo "Árbitro".

- Si hay **varias personas**, elegir como objetivo a la que **está mirando al robot**.
- Si **ninguna** lo mira, elegir una de ellas (criterio documentado: la más cercana, la más centrada, etc.) y centrar la mirada en ella.
- El cambio de objetivo debe tener **histéresis**: el robot no debe saltar entre personas continuamente; sólo cambia si el nuevo candidato es mejor durante un tiempo mínimo.

T5. Comportamientos

Una vez que tenemos el mecanismo, el seguidor y una primera versión del "Árbitro", empecemos por implementar comportamientos. La Tabla 1 resume algunos de los comportamientos propuestos.

Comportamiento	**Disparador**	**Respuesta esperada**
Observador	Sin personas	Centra la vista y explora la escena: barridos lentos, sacadas (cambios rápidos de mirada) a puntos de interés, pequeñas variaciones de roll para "encuadrar". Nada de movimiento aleatorio brusco.
Seguimiento (eye contact)	`PERSONA`	Sigue a la persona objetivo con la mirada (T3 y T4).
Saludo	`SALUDO`	Asiente con la cabeza (2–3 cabeceos en pitch) y emite un sonido tipo R2D2 (trinos y barridos de frecuencia).
Hincha de Olimpia	`CAMISETA_OLIMPIA`	Silba un fragmento reconocible de la polca de Olimpia **mientras sigue a la persona con la mirada**.
Hincha de Cerro	`CAMISETA_CERRO`	Emite un silbido de reprobación (p. ej., glissando descendente largo, "abucheo").

Tabla 1: Comportamientos requeridos.

También tenemos los modos de personalidad (seleccionables por comando). Éstos modifican la reacción del robot cuando sigue a una persona. Deben poder cambiarse en tiempo de ejecución mediante un comando (teclado, puerto serie, interfaz gráfica, etc.; p. ej. `MODE TIMIDO`), y el robot debe confirmar el cambio con un sonido breve distintivo.

- **Tímido:** si el robot sigue a una persona y ésta **lo mira** (`ME_MIRA`), el robot **desvía la mirada** (gira la cabeza hacia un costado y/o abajo), la mantiene desviada unos segundos y luego vuelve a "espiar" gradualmente.
- **Atrevido:** si el robot mira a una persona y ésta lo mira, sostiene la mirada y hace un **gesto de enfrentamiento elevando periódicamente el mentón** (oscilación en pitch hacia arriba), sin perder el seguimiento en yaw.
- **Maleducado:** sigue a la persona con la mirada y, **cuando ésta va saliendo del cuadro** (`SALIENDO`), le dedica un silbido de piropo (una vez por salida).

Se recomienda además un modo **Neutro** (por defecto) en el que el robot simplemente sostiene el contacto visual.

T6. Arbitraje, reactividad y robustez

Éste es un requisito central del trabajo. El robot **no debe comportarse de forma errática** ni superponer acciones. Como mínimo se exige:

a) **Recursos exclusivos.** El cuello y el buzzer son recursos; en cada instante cada recurso tiene **un solo dueño**. Dos comportamientos pueden coexistir sólo si usan recursos distintos (p. ej., la polca usa el buzzer mientras el seguimiento usa el cuello).

b) **Prioridades.** Definir y documentar una tabla de prioridades. Compartimos el siguiente ejemplo:

Prio.	Comportamiento	Recursos	Interrumpible
0	Comando del operador (`STOP`, cambio de modo)	todos	—
1	Saludo (asentir + R2D2)	cuello + buzzer	no (acción corta)
2	Reacción a camiseta (polca / reprobación)	buzzer	sólo por 0–1
3	Reacción del modo (tímido / atrevido / maleducado)	cuello y/o buzzer	sí
4	Seguimiento (eye contact)	cuello	sí
5	Observador	cuello	sí

c) **Anti-repetición.** Periodos refractarios (*cooldown*) por persona y por comportamiento: p. ej., la polca no se repite para la misma persona antes de 20–30 s; el saludo no se responde dos veces por el mismo gesto ni se piropea a más de una persona cada 40 segundos.

d) **Anti-oscilación.** Filtros temporales en los eventos, histéresis en umbrales y tiempos mínimos de permanencia en cada estado (por ejemplo, una persona con camiseta de Olimpia no se define por un cuadro en el que aparezca y desaparezca, sino por, p. ej., 6 cuadros seguidos en los que se la ve con la camiseta para detectar su presencia, y 6 seguidos sin la camiseta para detectar su ausencia).

e) **Reactividad.** Percepción a ≥ 10 Hz; latencia desde el evento hasta el inicio de la respuesta ≤ 300 ms (orientativo). El sonido y los movimientos deben ser **no bloqueantes** (sin `delay()`; máquinas de estado temporizadas con `millis()` o equivalente).

f) **Recuperación.** Tras finalizar o interrumpir una acción, el robot debe volver de forma suave al comportamiento que corresponda (seguir u observar).

T7. Monitoreo

Mostrar en pantalla (PC o Raspberry Pi) el video con superposición (*overlay*) de: personas detectadas con su ID, persona objetivo, eventos activos (`ME_MIRA`, `SALUDO`, camiseta), modo actual, comportamiento que controla cada recurso y ángulos comandados. Registrar (*log*) las transiciones de estado con marca de tiempo.

6. Evaluación

La evaluación se realiza mediante una **demostración funcional presencial** del prototipo, en la fecha del Segundo Parcial.

6.1. Entregas, documentación y fechas

Documetación y entregables:

- Informe técnico (Cuaderno Colab): mecanismo y electrónica, modelos de IA usados y definición operativa de cada evento, arquitectura, tabla de prioridades, diagramas de estado de los modos, parámetros (umbrales, cooldowns, ganancias) y resultados.
- Código fuente completo y comentado, con `README` de instalación y ejecución.
- Video de demostración (≤ 3 min) mostrando todos los comportamientos.
- **Demo:** funcional, con alimentación propia, listo para la prueba.

Fecha:

- Demo: Fecha del **Segundo Parcial** en el LAR.

- **Archivos:** incluyendo informe en Cuaderno Colab, código fuente (`.zip` y enlace a repositorio), video, y opcionalmente archivos CAD del mecanismo. Fin de semana posterior al segundo parcial (Domingo).

7. Observaciones adicionales

- La iluminación de la escena y las camisetas para la demo son responsabilidad de cada grupo; el evaluador podrá además usar otras prendas para verificar falsos positivos.
- No se deben almacenar imágenes de personas sin su consentimiento; los datos de entrenamiento propios deben ser de los integrantes o de voluntarios que hayan aceptado.
- El modo "maleducado" y los silbidos de hinchada se limitan a sonidos del buzzer, en tono lúdico; no se admite contenido ofensivo.
- Se sugiere Python + OpenCV + MediaPipe/YOLO para la percepción y Arduino/ESP32 para los actuadores, pero la elección tecnológica es libre.

8. Referencias

[1] H. Kozima, M. P. Michalowski y C. Nakagawa, "Keepon: A playful robot for research, therapy, and entertainment", *International Journal of Social Robotics*, vol. 1, n.º 1, pp. 3–18, 2009. doi:10.1007/s12369-008-0009-8.

[2] C. Breazeal, *Designing Sociable Robots*. Cambridge, MA: MIT Press, 2002 (robot Kismet).

[3] R. A. Brooks, "A robust layered control system for a mobile robot", *IEEE Journal of Robotics and Automation*, vol. 2, n.º 1, pp. 14–23, 1986. doi:10.1109/JRA.1986.1087032.

[4] R. C. Arkin, *Behavior-Based Robotics*. Cambridge, MA: MIT Press, 1998.

[5] C. Lugaresi *et al.*, "MediaPipe: A framework for building perception pipelines", arXiv:1906.08172, 2019.

[6] J. Redmon, S. Divvala, R. Girshick y A. Farhadi, "You Only Look Once: Unified, real-time object detection", en *Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 779–788, 2016.

[7] J. Duque-Domingo, J. Gómez-García-Bermejo y E. Zalama, "Gaze control of a robotic head for realistic interaction with humans", *Frontiers in Neurorobotics*, vol. 14, art. 34, 2020. doi:10.3389/fnbot.2020.00034.

[8] F. Cid, J. Moreno, P. Bustos y P. Núñez, "Muecas: A multi-sensor robotic head for affective human robot interaction and imitation", *Sensors*, vol. 14, n.º 5, pp. 7711–7737, 2014. doi:10.3390/s140507711.